Tractor Trace Format Description.

Files:

1. App (tractor_t.app[.gz])
2. Common (tractor_t.com.<rank>[.gz])
3. PTPD output (ptpd-tractor_t.log)
4. Maps (maps.tractor_t.com.<rank>)
5. Main (tractor_t.mpi.<rank>[.gz])
6. Hardware (counters) file (tractor_t.hwc.<rank>[.gz])

Format descriptions

HWC Format description:

1. Each record is a single line. Field separator is ':'.
2. Each record corresponds a single mpi call with one exception: mpi_sendrecv and mpi_sendrecv_replace. These calls occupy 2 records.
3. The format is as follows:

   uniq_id:thread_id:TSCs_section:address_section:hardware_counters_section: memprofile_section

   a. uniq_id. Decimal, unsigned, started form 0, incremental. Unique id of the record.

   b. thread_id. Decimal, unsigned. Openmp thread number. If no openmp involved = 0;

   c. TSC_Section: a ':' separated values. Number of values are taken from num_tscs read from app file. All numbers are relative: 1st value to 1st one from previous record, rest ones to the 1st one of the current record (except for collective transfers where predictor is NOT updated). Pseudocode is as follows(tsc_val_base=0 at the very beginning):

```
for (int tscs_ind=0;tscs_ind<num_tscs;tscs_ind++) {
  relative_tsc_val=read_val( 2 + tscs_ind);
  real_tsc_val[tscs_ind]= relative_tsc_val + tsc_val_base;
  if (!is_coll_transfer()) tsc_val_base = real_tsc_val[0]; // save it for next iteration
}
```

d. Address_section: a':' separated values. Number of values are taken from num_addrs read from app file (see below). The values here are the IDs of real addresses which may be read from app file. Pseudocode is as follows:

```
for (int addr_ind=0;addr_ind<num_addrs;addr_ind++) {
   addr_id=read_val( 2 + num_tscs+addr_ind);
}
```

e. Hardware_counters_section is not fully described here (since its usage is very narrow). It's a ':' separated section particularly with 4 values (2 pairs): instruction_retired and clockticks before mpi call (i.e. in user section preceding mpi call) and during mpi call.

f. Memprofile_section is NOT described here.

Main MPI Format description:

4. All MPI functions are divided into groups called classes. Now it's 9 classes.
    a. P2P (point to point (or peer to peer))
    b. PROBE (mpi_probe etc)
    c. REQUEST (MPI_WAIT/TEST etc)
    d. COLLECTIVE (MPI_BCAST etc)
    e. COMM or COMM_COLLECTIVE (communicator) MPI_COMM_CREATE etc.
    f. OSC (One Sided Communication) MPI_GET/PUT/ACCUMULATE etc
    g. FIO (File IO)
    h. BUFFERED_LOCAL (MPI_PACK/UNPACK)
    i. LOCAL (all other)
5. record divided into 4 parts: fixed, class dependent, flexible (function/parameter dependent), extra (special tractor build dependent)
6. Field separator is ':'. Extra part can be delimited by ';' as well.
7. Fixed part

    uniq_id:thread_id:mpi_func_id:

    a. uniq_id. The same as above.

b. thread_id. The same as above.

c. mpi_func_id – decimal, unsigned. Unique ID of MPI function[1].

8. Class dependent and flexible

   a. P2P and PROBE

      Class dependent part.

      :comm:peer:request:message:tag:transfer_bytes

      1. comm – decimal, signed. Global MPI Communicator ID.
      2. peer – decimal, signed. Remote rank (number is in MPI_COMM_WORLD). Can contain wildcards (MPI_ANY_SOURCE) and MPI_PROC_NULL.
      3. request – hexadecimal, unsigned. MPI Request. Valid only in case of non-blocking send/receive or persistent request. For MPI_IPROBE there is flag here. For other P2P and PROBE class function its value should be 0.
      4. message– hexadecimal, unsigned. MPI Message. Valid only in case of send/receive/probe with message (MPI_MXXXX,MPI_ImXXXX, available in MPI3).
      5. tag – decimal, signed. MPI TAG. Can contain wildcards (MPI_ANY_TAG).
      6. transfer_bytes – decimal, signed. Number of transferred bytes[2].

      There is no flexible part in case of P2P and PROBE class.

   b. REQUEST

      Class dependent part.

      : input_request_num:triggered_request_num:flag:num_colts:request_addr

      1. input_request_num – decimal, signed. Number of input requests.
      2. triggered_request_num – decimal, signed. number of request which complete during this call.

1If T_PRINT_FUNC_NAMES is turned on this value is the string representing actual MPI call (like MPI_INIT).

2Meaningful only in case of actual sending or receiving. In probes its value is undefined. In case of persistent request (MPI_XXXX_Init) it shows actual value of sending or receiving bytes of corresponding MPI_STARTXXX call.

3. flag – decimal, signed. flag for MPI_TEST where applicable and -1 for any other case.

4. num_colts – decimal, signed. Number of mpi calls which initiated by MPI to handle this collective. If collective splitting is unsupported by MPI num_colts =0 ;

5. request_addr - hexadecimal, unsigned. Address of MPI_Requests(S).

Flexible part:

:v_0:v_1:…:v_n – hexadecimal, unsigned. Values of each request supplied to the function. N=input_request_num.

:t_{00}:t_{01}:t_{02}:t_{03}…:t_{m0}:t_{m1}:t_{m2}:t_{m3} – 4-tuple representing[3] (index(decimal, signed),tag(decimal, signed),source(decimal, signed),transferred_bytes(decimal, signed)) of each completed request. m=triggered_request_num. index=[0…input_request_num-1]

c. COLLECTIVE

Class dependent part.

:comm:root:request:send_type_sizes_num:recv_type_sizes_num:
send_counts_num:recv_counts_num:num_colts

1. comm – decimal, signed. Global MPI Communicator ID.

2. root – decimal, signed. Root process(in MPI_COMM_WORLD). -1 if no root process in this particular call.

3. send_bytes_num, recv_bytes_num - decimal, signed. Number of send and receive bytes fields.

4. Request - hexadecimal, unsigned. MPI_Request if applicable, 0 otherwise.

5. num_colts - decimal, signed. Number of mpi calls which initiated by MPI to handle this collective. If collective splitting is unsupported by MPI num_colts =0 ;

Flexible part:

:SB_0:SB_1:…:SB_p – decimal, signed. Bytes sent by this rank. p=send_bytes_num. Followed by :RB_0:RB_1:…:RB_q – decimal, signed. Bytes received by this rank. q=recv_bytes_num.

d. COMM

Class dependent part.

:comm::newcom:remote_comm:inter_comm

3Valid only for “receive” requests (f.i. initiated by mpi_irecv or mpi_recv_init). For send or null requests their value are undefined and should be ignored.

All 3 are decimal, signed and equal to 0 if not applicable (i.e. not all 4 exist in all communicator functions). No flexible part.

e. OSC

Class dependent part is pretty similar to P2P. All values have the same meanings except explicitly stated otherwise.

:comm:peer:request:window:size:transfer_bytes

1. size – decimal, signed. Size for window creation functions. Value should be ignored for all others.
2. window – hexadecimal, unsigned. MPI window handle.

This class has no flexible part.

f. FIO

Class dependent part.

For all functions except MPI_FILE_OPEN it's:

:offset:fh:request:transferred_bytes where

1. offset – decimal, signed. Offset if file if applicable. 0 otherwise.
2. fh – hexadecimal, unsigned. File handle.
3. request – hexadecimal, unsigned. MPIO_Request if applicable. 0 otherwise.
4. transferred_bytes – decimal, signed. Number of transferred bytes if applicable. 0 otherwise.

For MPI_FILE_OPEN class dependent part is:

:comm:fh

Meanings are the same as above.

g. Buffered Local.

:count:size

1. count – decimal, signed.
2. size – decimal, signed.

h. Local has no class dependent and flexible parts.

Extra part:

Printed at the end of flexible part. Now only 3 extra parts are defined:

1. Buffers addresses (only where applicable).
    a. Printed after ';'.
    b. Field separator is ':'.
    c. Can contain 4-tuples (virtual_address,size,physical_entry,flags) if VIRT2PHYS is enabled or 2-tuples (virtual_address,size) if MPIBUFFERS is enabled.
    d. If particular tuple is invalid all its values are zeros.

MAP Format description:

Copy of /proc/pid/maps

Rest files format description

These files contain just records in flexible format (i.e. format depends on record number).

RECORD_ID:RECORD.

App Format description:

PEAT File Record Type Definitions.
An informative table to make correlation between id and theirs "high level" meaning. Can be used to translate messages between different version of tractor and pet/tv. (1 = PEAT_FRT_FRT_Definitions)
1:id:description (PEAT_FRT_)

PEAT_FRT_FRAME_MARKER:decimal_value – currently not used.
PEAT_FRT_VarBlockInfo:num_tscs:num_addrs:num_hwcs – number of values used to decode hwc file.
PEAT_FRT_MPMMasterRanks:number:ranks – Master (first rank on each node) ranks. Number is decimal nonnegative value. ranks are separated by ":".
PEAT_FRT_MPMNodesFreq:node_num:rank:freq0:freq1:init_delay:hostname. All values are decimal except hostname (string). Init_delay is now not used.
PEAT_FRT_MPMRanksAndPPN:hostname:frequency:ppn:id:ranks - mostly informational. Ranks separated by ','.
PEAT_FRT_MPMAllRanksTimeStamp:nodes_num:4_tuples – Initial time (ptp) correction for all nodes. number of 4 tuples = nodes_num. (Deprecated).

PEAT_FRT_MPIFunctionNameId:id:MPI_FUNC – translation between function id (decimal) and function name (string)
PEAT_FRT_GlobalCommunicatorsReport:id:size:ranks – this id is used in comm and p2p records.
PEAT_FRT_Sync_Point:size:<values> - Synchronization point. To be used if works without PTPD. Usually occurs twice (at the beginning and end of trace time).

Common Format description:

PEAT_FRT_AppStartStopInfo:<start_tscval>:<stop_tscval>:<duration>
PEAT_FRT_CpuFreq:0:<freq>:<freq>
PEAT_FRT_NumberOfEvents:number_of_events – decimal
PEAT_FRT_CollectorFrameInfo: - currently reserved (not used)
PEAT_FRT_UserAddresses:id:bin_address:binary_file:function:source_file:line
id, line – decimal
bin_addres – hexadecimal

All the rest – strings

PEAT_FRT_MPIPredefined:MPI_SUCCESS:MPI_UNDEFINED:MPI_COMM_WORLD: MPI_COMM_SELF: MPI_COMM_NULL: MPI_PROC_NULL:MPI_ANY_TAG:MPI_ANY_SOURCE:MPI_REQUEST_NULL

PTPD Format description:

PEAT_FRT_PTPMode:<dedicated_mode_in_use> (1 or 0)
PEAT_FRT_PTPRanks: <number_of_ranks>:<rank_to_comm_world_mappings> - 'masters' rank mapping record

PEAT_FRT_PTPNodesFreq: :<rank_id>:<freq1>:<freq2>:<hostname> - frequency and hostname record
PEAT_FRT_PTPMainRecord:<destination_rank>:t1:t2:t3:t4 - basic PTP record.